# CYBR 437 Secure Coding
# Winter 2026
# Lab - 5

Release date: February 3, 2026

**Due date: February 9, 2026, at 11:59 pm**

## Introduction

In this lab, you will gain hands-on experience analyzing and exploiting buffer overflow vulnerabilities. You will execute the following buffer overflow attacks, of escalating complexity, to gain insight into the stack's operation and how insecure code can lead to data corruption, control flow manipulation, and memory access violations.

## Setup

- A lab5 tarball is available for download from the Lab-5 assignment page, within are 4 subfolders named task 1-4. These subfolders contain a C source file, a flag text file, and an *exploit.py* template.
- Upload these tarballs to your VM and untar with the following commands:
  - `scp ./lab5.tar.gz [username]@[ip]:./[directory]`
  - `tar –xzvf lab5.tar.gz`
- Your task is to modify the second parameter of the `sendlineafter()` function in *exploit.py* to craft the required input for the buffer overflow exploit.
  - You will only change the contents within the parentheses for `sendlineafter` for the second and third (if the third argument exists) arguments.
  - Once you have modified the contents of `sendlineafter`, save and execute the script with `python3 exploit.py`
  - For this, you will need to install pwntools using the command:
    `sudo apt install python3-pwntools`
- Compile the source file for task 1 through task 4 with the following command:
  `gcc *.c -o output -fno-stack-protector -z execstack -no-pie -m32`
- Note: You may not modify the C code. Changing the C code will result in a score of 0 for the task.

## Tasks (25 points per task)

**Task 1**

Overview: This task introduces basic buffer handling and conditional logic. Modify the input in *exploit.py* to overflow the buffer and obtain the flag.

Hints:

• Understand the structure of the program by examining the source code.

• Incrementally increase the input size to determine the buffer's capacity.

**Task 2**

Overview: This task, similar in structure to the first, focuses on user input and conditional checks against a specific variable's value. It introduces a slightly different scenario for the conditional check.

Hints:

- Understand the structure of the program by examining the source code.
- Endianness matters – ensure proper byte order in your input. Check your binary endianness with the file command.

**Task 3**

Overview: This task introduces function calls and validation conditions, focusing on EIP (Extended Instruction Pointer) manipulation.

Hints:

- The EIP is crucial for controlling execution flow.
- Identify function memory addresses using GDB or objdump.
- Utilize the cyclic command as demonstrated in class.

**Task 4**

Overview: This task focuses on overwriting function parameters to manipulate control flow and bypass validation.

Hints:

- Parameters are stored on the stack in most calling conventions.
- Debugging tools like pwngdb are essential for locating parameters and crafting the exploit.

## Submission Instructions

Your write-up submission should include:

- For every problem, screen captures demonstrating:
  - Checking your file's endianness with the file command.
  - Checking what has been turned on/off with `checksec`.
  - Use of pwndbg to illustrate how you arrived at the value or the eip.
  - Your success in each task (screenshots must include the entire output from running *exploit.py*).
- Detailed explanations of how you performed each buffer overflow attack, including tools and techniques used.
- The flag associated with each buffer overflow attack.